

Семинар №9

Шаблоны: вывод типов,
structured bindings, tuples

auto

- Компилятор автоматически выводит тип на основе выражения
- Правила для вывода типов (type deduction) для auto такие же, как и для шаблонов, включая правила работы со ссылками, указателями и константностью
- auto&& - тоже универсальная ссылка
- Можно использовать в возвращаемом значении (можно указать возвращаемый тип через стрелочку)
- Можно использовать в качестве типа аргумента (эквивалентно шаблону функции)
- Reference collapsing

decltype

- Замена в CompileTime на тип выражения в скобках
- Тип точный, не отбрасывает ссылки (в отличие от auto)
- В том числе, decltype умеет отличать ссылки от самих объектов
- Reference collapsing
- Для expression количество & зависит от категории выражения
- decltype(auto) – вывести тип по правилам decltype, а не auto

Deducing This

- Позволяет выводить `this` по стандартным правилам `type deduction`
- Можно использовать при наследовании, тогда тип выведется как у наследника
- Это позволяет облегчить CRTP
- Более подробно можно прочитать [на Хабре](#)

CTAD и structured bindings

- CTAD – class template argument deduction
- Позволяет компилятору самому вывести шаблонный параметр объекта класса
- Можно задать собственные [deduction guides](#)
- [Structured bindings](#) – распаковка элементов
- Работает для:
 - Array
 - Любой класс, реализующий tuple operations (tuple_size, tuple_element)

std::tuple

- Кортеж элементов, обобщение пары на переменное количество элементов
- `std::get<int>` / `std::get<1>`
- Нужно задать `deduction guides` для вывода типов в конструкторе от пар и аллокаторов

Способы создания `std::tuple`

- Вызов конструктора
- `std::make_tuple` – можно не указывать явно параметры (не очень полезно из-за CTAAD)
- `std::make_tuple` применяет `std::decay` к типам -> теряем ссылочность и константность (можем решить `std::ref`, но не лучшее решение)
- `std::tie` – добавляет ссылочность своим параметрам
- `std::forward_as_tuple` – применяет правила универсальных ссылок для вывода типов
- Подробнее здесь

std::ignore

- Объекту `std::ignore` класса `std::ignore_t` можно присваивать все, что угодно
- Удобно использовать для игнорирования при распаковке с помощью `std::tie`
- Можно игнорировать `[[nodiscard]]`

Реализация `std::tuple`

- Разбиваем на `Head` и `Tail`, храним в полях `Head` и `Tail` или наследуемся от `tuple<Tail...>`
- `get` – не метод (иначе пришлось бы писать `t.template get<>`)
- Для реализации методов опять разделяем на голову и хвост